



សាកលវិទ្យាល័យ ភូមិន្ទភ្នំពេញ

Royal University of Phnom Penh

មហាវិទ្យាល័យ : វិទ្យាសាស្ត្រ

ជេប៉ាតីម៉ង់ : ព័ត៌មានវិទ្យា

Database Systems Design

បង្រៀនដោយ ៖ លោកគ្រូ Kheam Hong

**បោយការណ៍ទី២ អំពី៖ ការប្រើប្រាស់View,
Stored Procedures and Trigger**

ក្រុមទី១៤

សមាជិកក្រុម ៖

កាន ចន្ទា

ខ្នង សាវ៉ែ



Contents

I. គោលបំណងមេរៀន	4
II. ការណែនាំអំពី View,Stored Procedures and Triggers.....	5
1. View	5
1.1. អត្ថប្រយោជន៍នៃ View (Advantages of Views)	5
1.2. View និងសុវត្ថិភាព (Views and Security)	6
1.3. View Syntax and Options	7
2. Stored Procedures	9
2.1. អត្ថប្រយោជន៍ (Advantages) របស់ Stored Procedures	9
o Disadvantages	10
2.2. Syntax of Stored Procedure	11
2.3. Stored Procedure for DELETE	12
Syntax :	12
CREATE PROCEDURE ProcedureName	12
@Parameter DataType	12
AS	12
BEGIN	12
DELETE FROM TableName.....	12
WHERE Condition;	12
END;.....	12
2.4. 4.Stored Procedure for UPDATE.....	12
Syntax	13

3. Triggers.....	13
3.1. Advantages of Triggers	14
Syntax:	14
3.2. ប្រភេទ នៃ Triggers.....	15
ក. AFTER Trigger.....	15
ខ. INSTEAD OF Trigger.....	15
III. ការអនុវត្ត	16
IV. Reflection	30
V. សេចក្តីសន្និដ្ឋាន	31
VI. Future Work(ទិសដៅការងារនាពេលអនាគត)	31

I. គោលបំណងមេរៀន

- ☞ បង្កើនប្រសិទ្ធភាពការងារនៅលើSQL
- ☞ បង្កើនប្រសិទ្ធភាពការងារលើ Server តាមរយៈផែនការប្រតិបត្តិការដែលបាន compiled ទុកជាមុន និងកាត់បន្ថយចរាចរណ៍ទិន្នន័យរវាង Server របស់ Database និងស្រទាប់កម្មវិធី (Application layers) ។
- ☞ អនុវត្តច្បាប់ និងការកំណត់កម្រិតស្វ័យប្រវត្តិនៃមូលដ្ឋានទិន្នន័យនៅលើDatabase
- ☞ ពង្រឹងការធ្វើតេស្តប្រព័ន្ធ និងការធានាគុណភាព (ទាំង Client-side និង Server-side)
- ☞ ការធានាសុវត្ថិភាពរឹងមាំលើមូលដ្ឋានទិន្នន័យ និងកម្មវិធី (Database & Application Security)

II. ការណែនាំអំពី View, Stored Procedures and Triggers

1. View

View គឺជាពាក្យបញ្ជា SELECT ដែលត្រូវបានរក្សាទុក (Saved SELECT statement) ។ អ្នកអាចគិតថាវាជា "តារាងនិម្មិត" (Virtual Table) ព្រោះវាអាចទាញយកទិន្នន័យពីតារាងមួយ ឬច្រើន ព្រមទាំងអាចទាញយកទិន្នន័យឆ្លងកាត់ Database ឬ Server ផ្សេងៗទៀតបាន។

1.1. អត្ថប្រយោជន៍នៃ View (Advantages of Views)

មូលហេតុដែលគេប្រើប្រាស់ View ក្នុងការអភិវឌ្ឍកម្មវិធី៖

- ភាពងាយស្រួល (Friendly Names) : អ្នកអាចដាក់ឈ្មោះជួរឈរ (Column) ក្នុង View ឱ្យងាយយល់ ដើម្បីឱ្យអ្នកប្រើប្រាស់ (User) ងាយស្រួលក្នុងការទាញយកទិន្នន័យ ដោយមិនចាំបាច់ស្គាល់ឈ្មោះជួរឈរដើមក្នុងតារាងនោះទេ។
- ការប្តូរតាមបំណង (Customization) : អ្នកអាចប្រើ View ដើម្បីបង្ហាញទិន្នន័យតាមតម្រូវការជាក់លាក់ ដូចជាការបោះ (Filter) ឬការតម្រៀប (Sort) ផ្សេងៗគ្នា សម្រាប់អ្នកប្រើប្រាស់ផ្សេងៗគ្នា ឬកម្មវិធីផ្សេងៗគ្នា។

- ការលាក់ភាពស្មុគស្មាញ (Hiding Complexity) : អ្នកប្រើប្រាស់ ឬកម្មវិធីដែលមកតភ្ជាប់ (Client Application) មិនចាំបាច់ដឹងពីភាពស្មុគស្មាញនៃ Query ដើម ដូចជាការប្រើ JOIN ឬការធ្វើ Aggregates (ដូចជា SUM, AVG) នោះទេ។ ពួកគេគ្រាន់តែដឹងថាពួកគេកំពុងហៅយកទិន្នន័យពី View មួយប៉ុណ្ណោះ។
- ការលាក់ទិន្នន័យ : អ្នកអាចមិនរាប់បញ្ចូល (omit) ទិន្នន័យដែលមិនចាំបាច់ ឬទិន្នន័យដែលសំខាន់ (sensitive data) ចេញពី View បាន
- ការបង្កើនប្រសិទ្ធភាព : ការដំណើរការទិន្នន័យកើតឡើងនៅលើ Server ហើយមានតែលទ្ធផលដែលទទួលបានប៉ុណ្ណោះដែលត្រូវបានផ្ញើត្រឡប់ទៅ Client វិញ
- Backward Compatibility : View អាចដើរតួជាចំណុចតភ្ជាប់សម្រាប់តារាងដែលលែងមាន ឬតារាងដែលមានការផ្លាស់ប្តូររចនាសម្ព័ន្ធ (schema)
- ការនាំចេញទិន្នន័យ : អ្នកអាចប្រើ View ដើម្បីនាំចេញទិន្នន័យទៅកាន់កម្មវិធីផ្សេងទៀត
- ការបែងចែកទិន្នន័យ (Partitioning) : វាអាចជួយក្នុងការបែងចែកទិន្នន័យឱ្យមានសណ្តាប់ធ្នាប់។
- សុវត្ថិភាព : និយមន័យនៃ View អាចត្រូវបានដាក់កូដសម្ងាត់ (encrypted) ដើម្បីកុំឱ្យអ្នកដទៃអាចអានបាន។

1.2. View និងសុវត្ថិភាព (Views and Security)

View ប្រើសម្រាប់គោលបំណងសុវត្ថិភាព ដើម្បីឱ្យកម្មវិធីអាចចូលប្រើទិន្នន័យបានដោយមិនចាំបាច់មានសិទ្ធិពេញលេញលើតារាងដើម

អាចដកហូតសិទ្ធិទាំងអស់នៅលើតារាងដើម ហើយផ្តល់សិទ្ធិឱ្យតែតាមរយៈ View ប៉ុណ្ណោះ ដែលធ្វើឱ្យអ្នកមានការគ្រប់គ្រងបានល្អិតល្អន់ជាងមុនលើទិន្នន័យរបស់អ្នក។

1.3. View Syntax and Options

ប្រព័ន្ធគ្រប់គ្រងមូលដ្ឋានទិន្នន័យ (DBMS) ភាគច្រើន ពិសេស SQL Server (T-SQL) មានទម្រង់ពាក្យបញ្ជា (Syntax) និងជម្រើស (Options) សម្រាប់បង្កើត និងកែប្រែ View ដូចខាងក្រោម៖

ក ទម្រង់ទូទៅ

ទម្រង់ស្តង់ដារនៃការបង្កើត View គឺចាប់ផ្តើមដោយពាក្យបញ្ជា CREATE VIEW តាមបន្ទាប់ដោយឈ្មោះ View និងសំណួរ SELECT ៖

```
CREATE [ OR ALTER ] VIEW [ schema_name . ] view_name [ ( column [ ,...n ] ) ] [ WITH  
<view_attribute> [ ,...n ] ] AS
```

```
select_statement
```

```
[ WITH CHECK OPTION ] [ ; ]
```

- CREATE [OR ALTER] VIEW: បង្កើត View ថ្មី ឬកែប្រែ View ដែលមានស្រាប់រួចជាស្រេចដោយមិនបាច់លុបវាចោល (Avoids dropping dependencies)។

- view_name: ឈ្មោះរបស់ View ដែលត្រូវបង្កើតឡើង (ត្រូវមានតែមួយគត់ក្នុង Database schema នោះ)។
- column_name: ការកំណត់ឈ្មោះ Columns ថ្មីសម្រាប់ View
- AS select_statement: ពាក្យបញ្ជា SELECT ដែលកំណត់ពីទិន្នន័យ ឬកិច្ចការដែលត្រូវទាញយកមកដាក់ក្នុង View។

ខ ជម្រើសនិងលក្ខណៈពិសេស (View Options & Attributes)

យើងអាចបន្ថែម Options ផ្សេងៗនៅខាងក្នុងផ្នែក <view_attribute> ដើម្បីបង្កើនសុវត្ថិភាព ឬប្រសិទ្ធភាពការងារ៖

➤ WITH ENCRYPTION

Encrypt កូដប្រភព (Source text) របស់ View នៅក្នុង System Catalog (sys.sql_modules) ដែលជួយការពារកុំឱ្យអ្នកប្រើប្រាស់ដែលគ្មានការអនុញ្ញាតមើលឃើញ ឬលួចយកនិយមន័យ SQL ខាងក្នុងនោះ។

➤ WITH SCHEMABINDING

ចងភ្ជាប់ View ទៅនឹងគ្រោងសាងសង់ (Schema) នៃតារាងមូលដ្ឋាន (Base tables) ខាងក្រោម។ នៅពេលដែលជម្រើសនេះត្រូវបានប្រើប្រាស់ តារាងមូលដ្ឋានមិនអាចត្រូវបានលុបចោល ឬកែប្រែដែលអាចប៉ះពាល់ដល់និយមន័យរបស់ View នោះឡើយ លុះត្រាតែ View ត្រូវបានលុបចោល ឬដកការចងភ្ជាប់ជាមុនសិន។ នេះក៏ជាតម្រូវការចាំបាច់ផងដែរសម្រាប់ការបង្កើត Indexed View។

➤ WITH CHECK OPTION

ធានាថា រាល់សេចក្តីថ្លែងការណ៍កែប្រែទិន្នន័យ (INSERT ឬ UPDATE) ដែលត្រូវបានប្រតិបត្តិប្រឆាំងនឹង View គឺអនុលោមតាមលក្ខខណ្ឌដែលបានបញ្ជាក់នៅក្នុងឃ្លា WHERE របស់ View នោះ។ នេះជួយទប់ស្កាត់មិនឱ្យទិន្នន័យធ្លាក់ចេញពីដែនកំណត់ដែលមើលឃើញរបស់ View ឡើយ។

2. Stored Procedures

Stored Procedure គឺជាបណ្តុំនៃពាក្យបញ្ជា SQL (SQL statements) ដែលត្រូវបានសរសេរទុកជាមុន និងរក្សាទុកនៅលើ Server មូលដ្ឋានទិន្នន័យ (Database Server) ។

វាប្រហាក់ប្រហែលនឹង Function ឬ Method ក្នុងភាសាកម្មវិធី (Programming Languages)ដែរ

ដែលអ្នកអាចហៅយកមកប្រើប្រាស់ដដែលៗនៅពេលណាដែលត្រូវការ។

អត្ថប្រយោជន៍ (Advantages) របស់ Stored Procedures

- បង្កើនល្បឿន (Improved Performance) : ដំណើរការលឿនជាង SQL Query ធម្មតា ព្រោះត្រូវបាន Compile និង Optimize ជាមុន។
- អាចប្រើឡើងវិញ (Code Reusability) : សរសេរម្តង ហើយអាចហៅប្រើបានច្រើនដង។
- កាត់បន្ថយការសរសេរកូដដដែលៗ (Reduced Code Duplication) : ជួយឱ្យកូដមានរបៀបរៀបរយ និងងាយស្រួលគ្រប់គ្រង។

- បង្កើនសុវត្ថិភាព (Enhanced Security) : អ្នកប្រើអាច Execute Procedure ដោយមិនចាំបាច់ចូលប្រើ Table ដោយផ្ទាល់។
- ងាយស្រួលថែទាំ (Easier Maintenance) : កែប្រែ Procedure តែមួយកន្លែង អាចអនុវត្តលើកម្មវិធីទាំងមូល។
- កាត់បន្ថយការប្រើប្រាស់បណ្តាញ (Reduced Network Traffic) : ផ្ញើតែពាក្យបញ្ជា Execute ទៅ Database ជំនួស SQL Query វែងៗ។
- គាំទ្រ Parameter (Parameter Support) : អាចទទួលបានតម្លៃ Input ដើម្បីអនុវត្តការងារផ្សេងៗបាន។
- គាំទ្រ Transaction (Transaction Management) : អាចប្រើ COMMIT និង ROLLBACK ដើម្បីធានាភាពត្រឹមត្រូវនៃប្រតិបត្តិការ។
- រក្សាភាពត្រឹមត្រូវនៃទិន្នន័យ (Improved Data Integrity) : អាចបន្ថែម Validation និង Business Rules ដើម្បីការពារទិន្នន័យមិនត្រឹមត្រូវ។

○ Disadvantages

ទោះបីជា Stored Procedure មានអត្ថប្រយោជន៍ច្រើនក៏ដោយ ក៏វាមានគុណវិបត្តិមួយចំនួនដែរ៖

- ការសរសេរ និង Debug អាចស្មុគស្មាញជាង SQL Query ធម្មតា។
- Procedure ដែលមានទំហំធំ ឬ Logic ស្មុគស្មាញ អាចពិបាកក្នុងការថែទាំ។
- Syntax របស់ Stored Procedure អាចខុសគ្នារវាង Database Management System នីមួយៗ ដូចជា SQL Server, MySQL និង Oracle។

Syntax of Stored Procedure

Stored Procedure ត្រូវបានបង្កើតដោយប្រើពាក្យបញ្ជា CREATE PROCEDURE ។

```
CREATE PROCEDURE procedure_name
```

```
    @parameter1 datatype [= default_value] [OUTPUT],
```

```
    @parameter2 datatype [= default_value] [OUTPUT] AS BEGIN
```

```
-- ក្នុង SQL / T-SQL របស់អ្នកនៅទីនេះ
```

```
SELECT column1, column2 FROM table_name
```

```
WHERE condition = @parameter1; END;
```

ដើម្បីដំណើរការ (Execute) Stored Procedure ប្រើ៖

```
EXEC ProcedureName;
```

- **CREATE PROCEDURE** (ឬអាចសរសេរកាត់ថា **CREATE PROC**) ៖ ជាពាក្យបញ្ជាប្រាប់ SQL Server ឱ្យចាប់ផ្តើមបង្កើត Stored Procedure ថ្មី
- **procedure_name** ៖ ជាឈ្មោះដែលយើងចង់ដាក់ឱ្យ Stored Procedure របស់យើង (ឧទាហរណ៍៖ GetEmployeeByID) ។
- **@parameter1, @parameter2** ៖ ជាប៉ារ៉ាម៉ែត្រ (Parameters) សម្រាប់បញ្ជូនតម្លៃចូល (Input) ឬទាញយកតម្លៃចេញ (Output) បើមាន។ ផ្នែកនេះជាជម្រើស (Optional) គឺមាន ឬអត់ក៏បាន។
- **AS** ៖ ជាពាក្យគន្លឹះកំណត់ចំណុចចាប់ផ្តើមនៃតួអក្សរកូដខាងក្នុង Procedure ។

- **BEGIN ... END:** ជាប្លុក (Block) កំណត់ព្រំដែននៃកូដ T-SQL ដែលត្រូវដំណើរការ (អាចមាន ឬអត់ក៏បាន ប្រសិនបើមានកូដតែ១បន្ទាត់ ប៉ុន្តែការប្រើប្រាស់ BEGIN...END គឺល្អនិងមានរបៀបរៀបរយជាងពេលមានកូដច្រើន)។

Stored Procedure for DELETE

Stored Procedure សម្រាប់ **DELETE** ត្រូវបានប្រើដើម្បីលុបទិន្នន័យចេញពីតារាង (Table) ដោយផ្អែកលើលក្ខខណ្ឌជាក់លាក់ ឬ Parameter ដែលអ្នកប្រើផ្តល់ឱ្យ។ ការប្រើ Stored Procedure សម្រាប់ការលុបទិន្នន័យជួយបង្កើនសុវត្ថិភាព និងធ្វើឱ្យការគ្រប់គ្រងទិន្នន័យមានភាពងាយស្រួលជាងការសរសេរ DELETE Query ដោយផ្ទាល់។

Syntax :

```
CREATE PROCEDURE ProcedureName
```

```
    @Parameter DataType
```

```
AS
```

```
BEGIN
```

```
    DELETE FROM TableName
```

```
    WHERE Condition;
```

```
END;
```

4. Stored Procedure for UPDATE

Stored Procedure សម្រាប់ **UPDATE** ត្រូវបានប្រើដើម្បីកែប្រែ (Update) ទិន្នន័យដែលមានស្រាប់នៅក្នុងតារាង (Table) ដោយផ្អែកលើលក្ខខណ្ឌ ឬ Parameter ដែលអ្នកប្រើផ្តល់ឱ្យ។ ការប្រើ Stored Procedure សម្រាប់ UPDATE ជួយធានាថាការកែប្រែទិន្នន័យមានភាពត្រឹមត្រូវ មានសុវត្ថិភាព និងងាយស្រួលក្នុងការគ្រប់គ្រង។

Syntax

```
CREATE PROCEDURE ProcedureName
    @Parameter1 DataType,
    @Parameter2 DataType
AS
BEGIN
    UPDATE TableName
    SET ColumnName = @Parameter1
    WHERE Condition;
END;
```

3. Triggers

Trigger គឺជា Object មួយនៅក្នុង Database ដែលដំណើរការដោយស្វ័យប្រវត្តិ (Automatically) នៅពេលមានព្រឹត្តិការណ៍ (Event) កើតឡើងលើ Table ឬ View ដូចជា INSERT, UPDATE, ឬ DELETE។ Trigger មិនចាំបាច់ហៅដោយប្រើ EXEC ដូចជា Stored Procedure ទេ ព្រោះវានឹងដំណើរការដោយស្វ័យប្រវត្តិនៅពេល Event ដែលបានកំណត់កើតឡើង។

Trigger ត្រូវបានប្រើដើម្បីត្រួតពិនិត្យទិន្នន័យ អនុវត្ត Business Rules កត់ត្រាប្រវត្តិការផ្លាស់ប្តូរទិន្នន័យ (Audit Log) និងការការពារកែប្រែទិន្នន័យដែលមិនត្រឹមត្រូវ។

- គោលបំណងសំខាន់របស់ Trigger គឺដើម្បីធ្វើឱ្យ Database អាចអនុវត្តសកម្មភាពដោយស្វ័យប្រវត្តិ នៅពេលមានការបន្ថែម កែប្រែ ឬលុបទិន្នន័យ។ វាជួយរក្សាភាពត្រឹមត្រូវ (Data Integrity) បង្កើនសុវត្ថិភាព និងកាត់បន្ថយការសរសេរកូដដដែលៗនៅក្នុងកម្មវិធី។

Advantages of Triggers

Trigger គេប្រើសម្រាប់៖

- ត្រួតពិនិត្យទិន្នន័យ (Validate Data) : ផ្ទៀងផ្ទាត់ថាទិន្នន័យត្រឹមត្រូវ មុន ឬ ក្រោយការកែប្រែ។
- កត់ត្រាប្រវត្តិ (Audit/Logging) : កត់ត្រាការបន្ថែម កែប្រែ ឬលុបទិន្នន័យ។
- អនុវត្តច្បាប់អាជីវកម្ម (Enforce Business Rules) : ធានាថាទិន្នន័យអនុវត្តតាមគោលការណ៍របស់ប្រព័ន្ធ។
- ធ្វើបច្ចុប្បន្នភាពទិន្នន័យដោយស្វ័យប្រវត្តិ : កែប្រែ ឬធ្វើបច្ចុប្បន្នភាព Table ផ្សេងៗ ដែលពាក់ព័ន្ធ។
- ការពារការកែប្រែខុស (Prevent Invalid Operations) : ទប់ស្កាត់ការបញ្ចូល កែប្រែ ឬលុបទិន្នន័យដែលមិនត្រឹមត្រូវ។
- ផ្ញើការជូនដំណឹង (Notifications) : អនុវត្តសកម្មភាព ឬជូនដំណឹងនៅពេលមានព្រឹត្តិការណ៍កើតឡើង។
- រក្សាភាពស្របគ្នានៃទិន្នន័យ (Maintain Data Consistency) : ធានាថាទិន្នន័យក្នុងតារាងផ្សេងៗមានភាពស្របគ្នា។

Syntax:

```
CREATE TRIGGER TriggerName
ON TableName
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    -- SQL Statements
END;
```

ប្រភេទ នៃ Triggers

ជាទូទៅ Trigger មាន 2 ប្រភេទ សំខាន់ៗ៖

ក. AFTER Trigger

AFTER Trigger គឺជា Trigger ដែលដំណើរការ **បន្ទាប់ពី** ពាក្យបញ្ជា INSERT, UPDATE ឬ DELETE បានអនុវត្តដោយជោគជ័យ។ វាត្រូវបានប្រើសម្រាប់ការកត់ត្រាប្រវត្តិទិន្នន័យ (Audit Log) ការជូនដំណឹង និងការអនុវត្ត Business Rules បន្ទាប់ពីទិន្នន័យត្រូវបានផ្លាស់ប្តូរ។

Example:

```
CREATE TRIGGER trg_AfterInsert
ON Student
AFTER INSERT
AS
BEGIN
    PRINT 'New student added successfully.';
END;
```

ខ. INSTEAD OF Trigger

INSTEAD OF Trigger គឺជា Trigger ដែលដំណើរការ **ជំនួស** ពាក្យបញ្ជា INSERT, UPDATE ឬ DELETE។ ជំនួសឱ្យការអនុវត្តពាក្យបញ្ជាដើម Trigger នឹងដំណើរការ Logic ដែលបានកំណត់។ វាត្រូវបានប្រើជាញឹកញាប់លើ View ឬនៅពេលចង់គ្រប់គ្រងការកែប្រែទិន្នន័យតាមលក្ខខណ្ឌពិសេស។

Example:

```
CREATE TRIGGER trg_InsteadDelete
```

```

ON Student
INSTEAD OF DELETE
AS
BEGIN
    PRINT 'Delete operation is not allowed.';
END;

```

III. ការអនុវត្ត

View

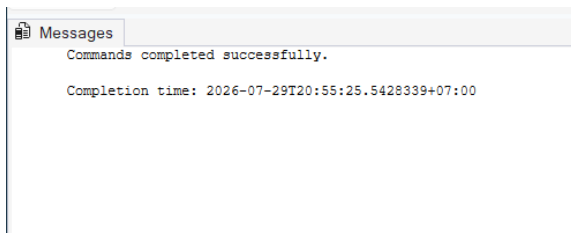
ឧទាហរណ៍ទី១ បង្កើត View មួយឈ្មោះថា vw_CustomerStates ដើម្បីទាញយកទិន្នន័យរួមគ្នារវាងតារាង Customers និងតារាង States ដោយបង្ហាញឈ្មោះអតិថិជន ទីក្រុង និងកូដរដ្ឋ (StateCode) ។

```

CREATE VIEW vw_CustomersStates AS
SELECT c.CustomerID, c.Name, c.City, s.StateCode
FROM dbo.Customers AS c
JOIN dbo.States AS s ON c.State = s.StateCode;
GO

```

▪ សារបញ្ជាក់ពី Server:



- ការទាញយកទិន្នន័យពី View (Querying the View): ដើម្បីពិនិត្យមើលទិន្នន័យដែលបានប្រមូលបាននៅលើ Server យើងបានធ្វើការហៅយកទិន្នន័យដោយប្រើប្រាស់ពាក្យបញ្ជា៖
SELECT * FROM dbo.vw_CustomersStates; GO

105 % ✓ No issues found

Results Messages

	CustomerID	Name	City	StateCode
1	1	Molunguri, A	Birmingham	AL
2	2	Muhinyi, Mauda	New York	NY
3	3	Antony, Abdul	Fayetteville	NC
4	4	Smith, Ahmad	Savanna	IL
5	5	Chen, Rabi	Murfreesboro	TN
6	6	Nuckols, Heather	Elgin	IL
7	7	Lutonsky, Christopher	Woodland Hills	CA
8	8	Reddy, Aj	Plano	TX
9	9	Rascano, Darrell	Rancho Cordova	CA
10	10	Johnson, Ajith	McGregor	CA
11	11	Patwardhan, John	Bethel Park	PA
12	12	Swenson, Vin	Albany	OR
13	13	Nichols, Nadine	Rogers	AR
14	14	Guthrie, Rawle	Phoenix	AZ
15	15	Nguyen, Allison	Memphis	TN
16	16	Kandregula, Howard	Oklahoma City	OK
17	17	Yodanis, Ed	Olney	KS

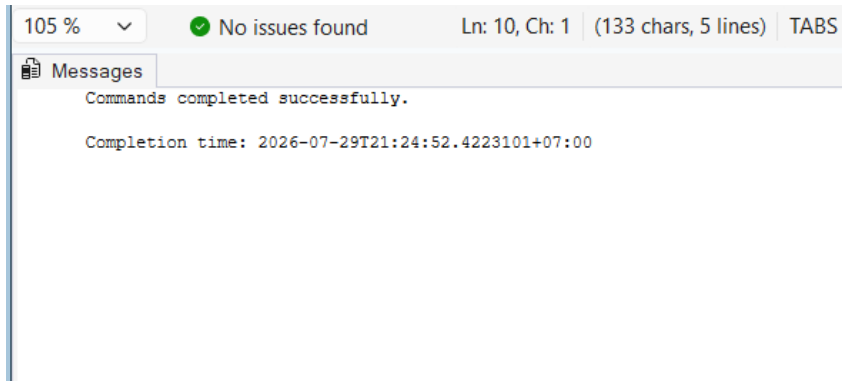
✓ Query executed successfully.

- លទ្ធផលខាងលើបង្ហាញពីការទាញយកទិន្នន័យដោយជោគជ័យចេញពី Server មកកាន់ Client តាមរយៈ View vw_CustomersStates ដែលធ្វើការ Join រវាងតារាង Customers និង States ដោយបង្ហាញព័ត៌មានអតិថិជន និងកូដរដ្ឋ (StateCode) យ៉ាងច្បាស់លាស់។

❖ **ឧទាហរណ៍ទី២** ការបង្កើត View បង្ហាញសរុបតម្លៃទំនិញក្នុងវិក្កយបត្រ (Invoice Summary)

```
CREATE VIEW vw_InvoiceSummary AS
SELECT InvoiceID, CustomerID, InvoiceDate, ProductTotal
FROM dbo.Invoices
Where ProductTotal>50;
GO
```

- សារបញ្ជាក់ពី Server:



➤ កូដហៅទិន្នន័យមកបង្ហាញ៖

```
SELECT *FROM dbo.vw_InvoiceSummary;
Go
```

	InvoiceID	CustomerID	InvoiceDate	ProductTotal
1	18	20	2013-01-13 00:00:00.000	155.00
2	23	694	2013-01-13 00:00:00.000	56.50
3	26	35	2013-01-13 00:00:00.000	56.50
4	27	11	2013-01-13 00:00:00.000	56.50
5	28	12	2013-01-13 00:00:00.000	56.50
6	29	13	2013-01-13 00:00:00.000	56.50
7	30	15	2013-01-13 00:00:00.000	56.50
8	31	11	2013-01-13 00:00:00.000	56.50
9	32	40	2013-01-13 00:00:00.000	113.00
10	33	33	2013-01-13 00:00:00.000	163.00
11	41	333	2013-01-13 00:00:00.000	56.50
12	42	666	2013-01-13 00:00:00.000	56.50
13	43	332	2013-01-13 00:00:00.000	56.50
14	44	555	2013-01-13 00:00:00.000	56.50
15	45	213	2013-01-13 00:00:00.000	56.50
16	46	20	2013-01-13 00:00:00.000	215.50
17	47	10	2013-01-13 00:00:00.000	226.50

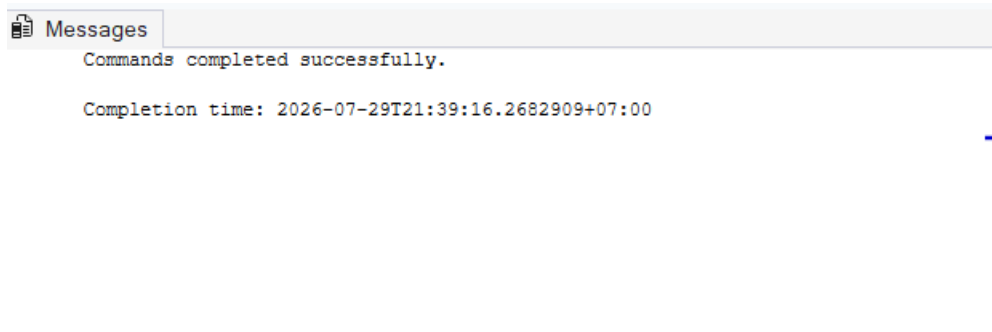
- ✓ រូបភាពនេះបង្ហាញពីដំណើរការទាញយកទិន្នន័យពី Server មកកាន់ Client តាមរយៈ View vw_InvoiceSummary ដែលបានតម្រងយកវិក្កយបត្រមានតម្លៃលើសពី ៥០ ដុល្លារ មកបង្ហាញជាតារាងយ៉ាងរលូន។

❖ **ឧទាហរណ៍ទី២** ការបង្កើត View បង្ហាញព័ត៌មានលម្អិតនៃការបញ្ជាទិញទំនិញ (Invoice Line Items Details)

```
CREATE VIEW vw_OrderDetails AS SELECT ili.InvoiceID, ili.ProductCode, p.Description, ili.Quantity,
ili.ItemTotal FROM dbo.InvoiceLineItems AS ili JOIN dbo.Products AS p ON ili.ProductCode =
p.ProductCode;
```

GO

▪ សារបញ្ជាក់ពី Server:



▪ កូដហៅទិន្នន័យមកបង្ហាញ៖ `SELECT * FROM dbo.vw_OrderDetails;`

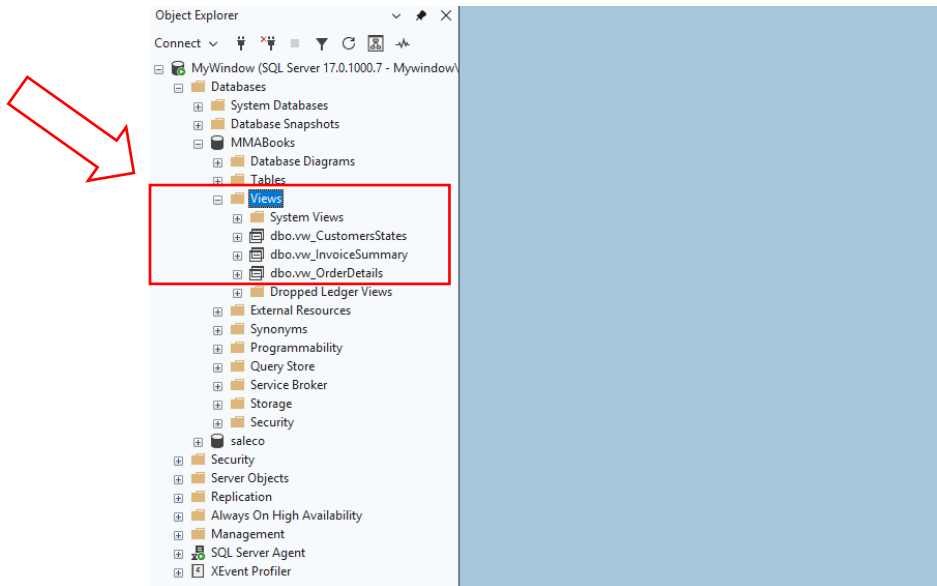
Go

	InvoiceID	ProductCode	Description	Quantity	ItemTotal
16	41	A4VB	Murach's ASP.NET 4 Web Programming with VB 2010	1	56.50
17	42	A4VB	Murach's ASP.NET 4 Web Programming with VB 2010	1	56.50
18	43	JAVP	Murach's Java Programming	1	56.50
19	44	A4VB	Murach's ASP.NET 4 Web Programming with VB 2010	1	56.50
20	45	A4VB	Murach's ASP.NET 4 Web Programming with VB 2010	1	56.50
21	46	A4VB	Murach's ASP.NET 4 Web Programming with VB 2010	1	56.50
22	46	DB2R	DB2 for the COBOL Programmer, Part 2 (2nd Edition)	1	45.00
23	46	SQ12	Murach's SQL Server 2012	1	57.50
24	46	VB10	Murach's Visual Basic 2010	1	56.50
25	47	A4VB	Murach's ASP.NET 4 Web Programming with VB 2010	1	56.50
26	47	DB2R	DB2 for the COBOL Programmer, Part 2 (2nd Edition)	4	180.00
27	48	A4VB	Murach's ASP.NET 4 Web Programming with VB 2010	1	56.50
28	49	A4VB	Murach's ASP.NET 4 Web Programming with VB 2010	1	56.50
29	49	DB1R	DB2 for the COBOL Programmer, Part 1 (2nd Edition)	1	42.00
30	50	A4VB	Murach's ASP.NET 4 Web Programming with VB 2010	1	56.50
31	57	A4VB	Murach's ASP.NET 4 Web Programming with VB 2010	1	56.50
32	58	A4VB	Murach's ASP.NET 4 Web Programming with VB 2010	1	56.50
33	59	A4VB	Murach's ASP.NET 4 Web Programming with VB 2010	10	565.00
34	60	A4VB	Murach's ASP.NET 4 Web Programming with VB 2010	1	56.50

192.168.100.135 (17.0 RTM) | Samai (82) | MMABooks | 00:00:00 | Row: 1, Col: 1 | 60 rows

- ✓ រូបភាពនេះបង្ហាញពីការទាញយកទិន្នន័យរួមគ្នា (Join) រវាងតារាងព័ត៌មានលម្អិតការបញ្ជាទិញ និងតារាងផលិតផលនៅលើ Server មកបង្ហាញជាតារាងដ៏សម្បូរបែបនៅលើ Client តាមរយៈ View vw_OrderDetails យ៉ាងមានប្រសិទ្ធភាព។

- ✓ លទ្ធផលនៅលើ Server (Server-Side Object Explorer):



- ✓ រូបភាពខាងលើបង្ហាញពីទិដ្ឋភាពក្នុង Object Explorer របស់ SQL Server ដែលបញ្ជាក់ថា View ទាំងបី (vw_CustomersStates, vw_InvoiceSummary, និង vw_OrderDetails) ត្រូវបានបង្កើត និងរក្សាទុកដោយជោគជ័យនៅក្នុង Database MMABooks លើ Server ស្របតាមគោលបំណងនៃការប្រើប្រាស់ View ក្នុងការប្រមូលផ្តុំ និងគ្រប់គ្រងទិន្នន័យយ៉ាងមានប្រសិទ្ធភាព។

Stored Procedure

- ❖ **ឧទាហរណ៍ទី១៖** ការបង្កើត និងហៅប្រើប្រាស់ Stored Procedure បង្ហាញព័ត៌មានផលិតផលតាមកូដ (sp_GetProductByCode)

```

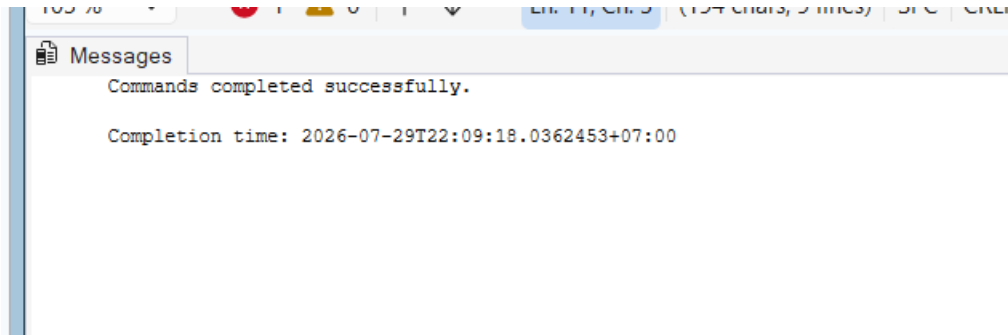
CREATE PROCEDURE sp_GetProductByCode
    @ProductCode VARCHAR(10)
AS
    
```

```

BEGIN
    SELECT ProductCode, Description, UnitPrice
    FROM dbo.Products
    WHERE ProductCode=@ProductCode;
END;
GO

```

▪ សារបញ្ជាក់ពី Server:



▪ ការហៅប្រើប្រាស់ (Execute) Stored Procedure

នៅពេលដែលចង់ទាញយកទិន្នន័យរបស់ផលិតផលណាមួយ អ្នកត្រូវបញ្ជូនតម្លៃ (Value) ទៅឱ្យ Parameter របស់វាដូចខាងក្រោម៖

```

EXEC sp_GetProductByCode @ProductCode='A4VB';
GO

```

The screenshot shows the 'Results' tab in a SQL Server interface. It displays a table with the following data:

	ProductCode	Description	UnitPrice
1	A4VB	Murach's ASP.NET 4 Web Programming with VB 2010	56.50

- ✓ **រូបភាពនេះ**បង្ហាញពីដំណើរការហៅប្រើប្រាស់ Stored Procedure (EXEC) ដោយបញ្ជូនតម្លៃ Parameter ('A4VB') ទៅកាន់ Server ដើម្បីទាញយកទិន្នន័យផលិតផលជាក់លាក់ណាមួយមកបង្ហាញជាតារាងនៅលើ Client យ៉ាងមានប្រសិទ្ធភាព និងសុវត្ថិភាពខ្ពស់។
- ❖ **ឧទាហរណ៍ទី២៖** ការបង្កើត Stored Procedure គណនា និងបង្ហាញស្ថានភាពវិក្កយបត្រ (sp_CheckInvoiceStatus)

```

CREATE PROCEDURE sp_CheckInvoiceStatus

    @InvoiceID INT

AS

BEGIN

    DECLARE @Total DECIMAL(9,2);

    -- យកតម្លៃ InvoiceTotal មកទុកក្នុង Variable

    SELECT @Total = InvoiceTotal

    FROM dbo.Invoices

    WHERE InvoiceID = @InvoiceID;

    -- ពិនិត្យលក្ខខណ្ឌប្រើ IF...ELSE (មានតែក្នុង Stored Procedure)

    IF @Total > 200

    BEGIN

        PRINT 'Invoice ID ' + CAST(@InvoiceID AS VARCHAR) + ' is a LARGE Invoice (Total: ' +
        CAST(@Total AS VARCHAR) + ')';

    END

    ELSE

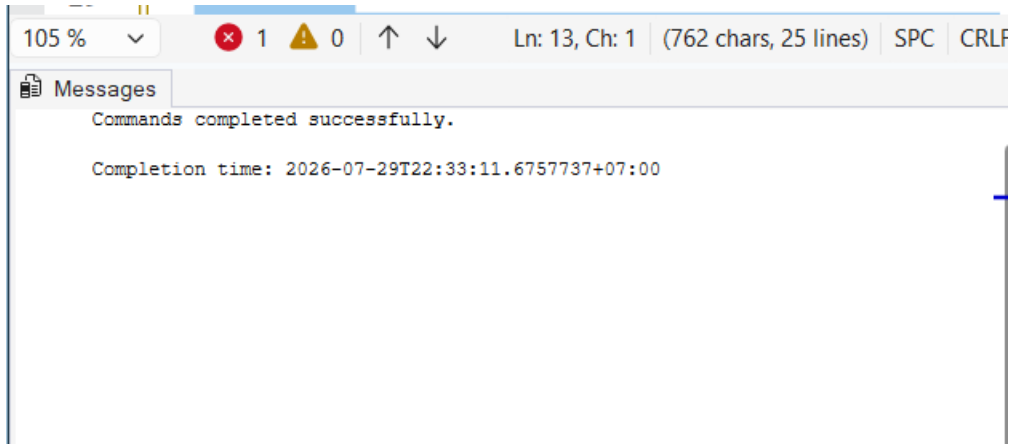
    BEGIN
  
```

```
PRINT 'Invoice ID ' + CAST(@InvoiceID AS VARCHAR) + ' is a NORMAL Invoice ( Total: ' +  
CAST(@Total AS VARCHAR) + '');
```

```
END;
```

```
GO
```

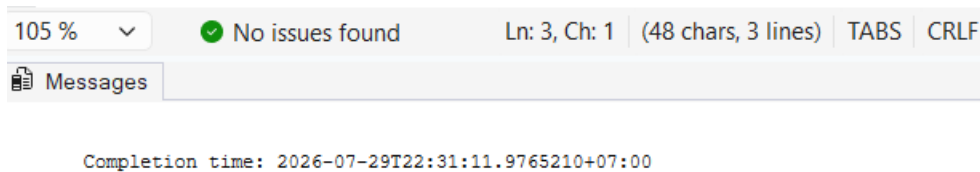
- សារបញ្ជាក់ពី Server:



- ការហៅប្រើប្រាស់ (Execute) Stored Procedure

```
EXEC sp_CheckInvoiceStatus @InvoiceID = 1;
```

```
GO
```



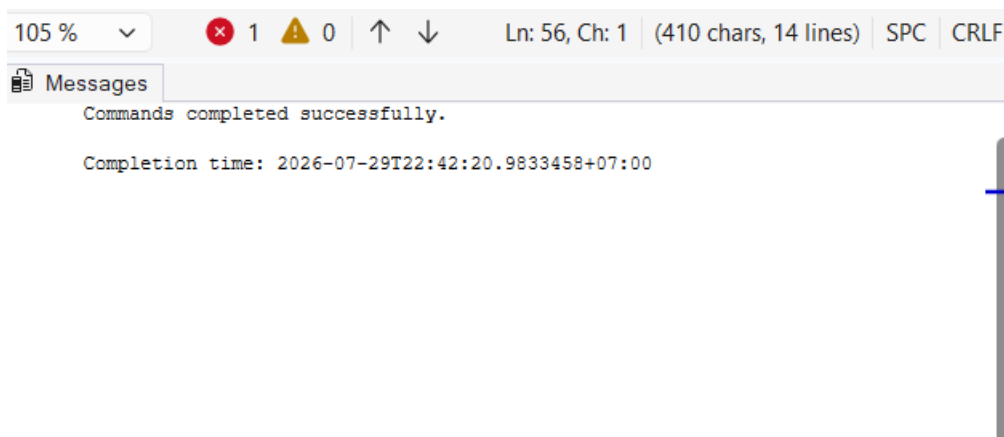
- ✓ បង្ហាញពេលវេលាបញ្ចប់ការដំណើរការ (Completion time: 2026-07-29T22:31:11.9765210+07:00) ដោយគ្មាន Error ពណ៌ក្រហមឡើយ ដែលបញ្ជាក់ថា Server បានទទួល និងដំណើរការពាក្យបញ្ជា PRINT ខាងក្នុង Stored Procedure បានយ៉ាងត្រឹមត្រូវ។

- ❖ **ឧទាហរណ៍ទី៣៖** ការបង្កើត Stored Procedure សម្រាប់កែប្រែតម្លៃផលិតផល (sp_UpdateProductPrice)

```
CREATE PROCEDURE sp_UpdateProductPrice
    @ProductCode VARCHAR(10),
    @NewPrice DECIMAL(9,2)
AS
BEGIN
    -- ធ្វើការ UPDATE តម្លៃថ្មីទៅកាន់តារាង Products
    UPDATE dbo.Products
    SET UnitPrice = @NewPrice
    WHERE ProductCode = @ProductCode;

    -- បង្ហាញសារជោគជ័យ
    PRINT 'Product ' + @ProductCode + ' price has been updated successfully to ' +
    CAST(@NewPrice AS VARCHAR);
END;
GO
```

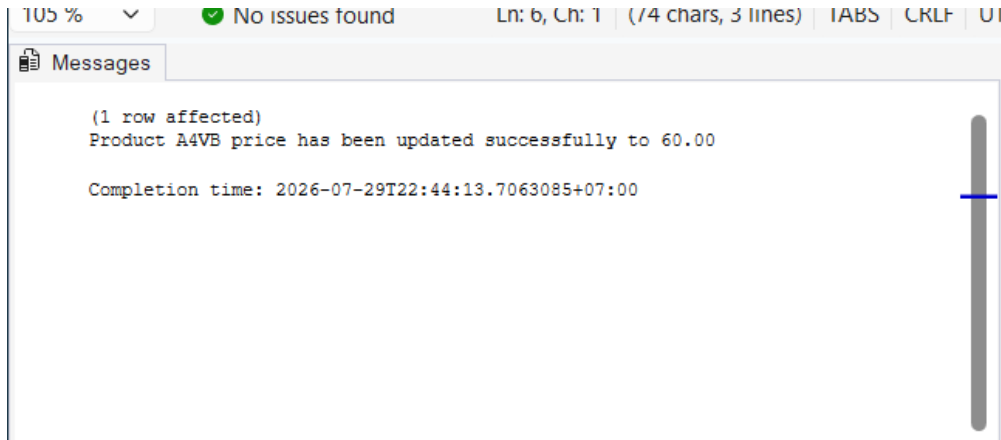
- សារបញ្ជាក់ពី Server:



- ការហៅប្រើប្រាស់ (Execute) Stored Procedure

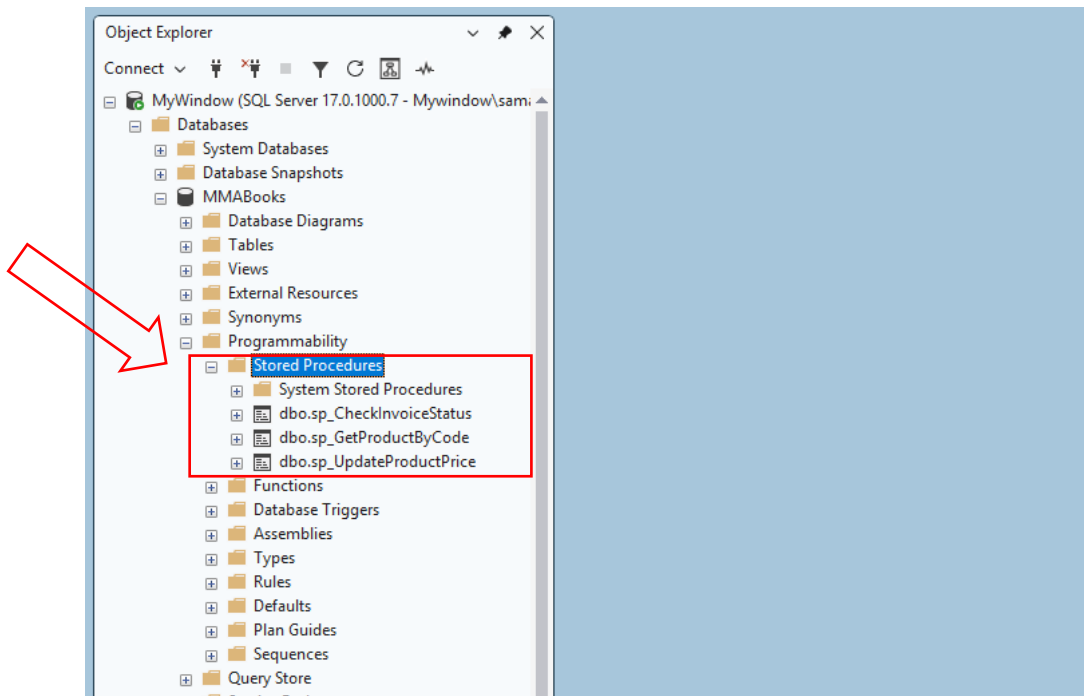
```
EXEC sp_UpdateProductPrice @ProductCode = 'A4VB', @NewPrice = 60.00;
```


GO



✓ **រូបភាពនេះ**គឺជាកត្តាតាងយ៉ាងច្បាស់ថា Stored Procedure អាចធ្វើការកែប្រែទិន្នន័យ (UPDATE) ដែលមានលក្ខណៈខុសប្លែក និងមានឥទ្ធិពលជាង View ធម្មតា។

❖ **លទ្ធផលនៅលើ Server**



- រូបភាពខាងលើបង្ហាញពីទិដ្ឋភាពក្នុង Object Explorer របស់ SQL Server ក្រោមថត Programmability > Stored Procedures នៃ Database MMABooks ដែលបញ្ជាក់ថា Stored Procedures ទាំងអស់ត្រូវបានបង្កើត រក្សាទុក និងត្រៀមខ្លួនរួចជាស្រេចសម្រាប់យកទៅហៅប្រើប្រាស់តាមតម្រូវការប្រព័ន្ធយ៉ាងមានប្រសិទ្ធភាព។

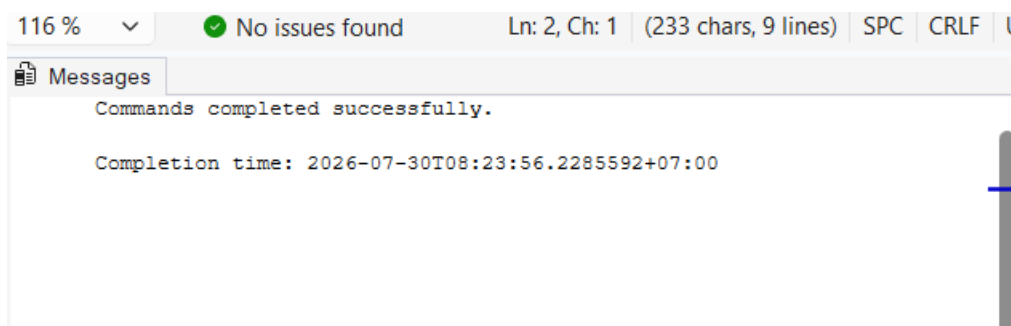
Trigger

ឧទាហរណ៍ទី១ បង្កើត Trigger ហាមឃាត់ ឬទប់ស្កាត់ការលុបទិន្នន័យអតិថិជន

បង្កើត Trigger ប្រភេទ INSTEAD OF DELETE ដើម្បីការពារកុំឱ្យមានការលុបព័ត៌មានអតិថិជន (Customers) ចោលដោយស្វ័យប្រវត្តិ នៅពេលដែលមានពាក្យបញ្ជា DELETE កើតឡើង ដើម្បីរក្សាសុវត្ថិភាពទិន្នន័យក្នុងប្រព័ន្ធ (trg_PreventCustomerDelete)

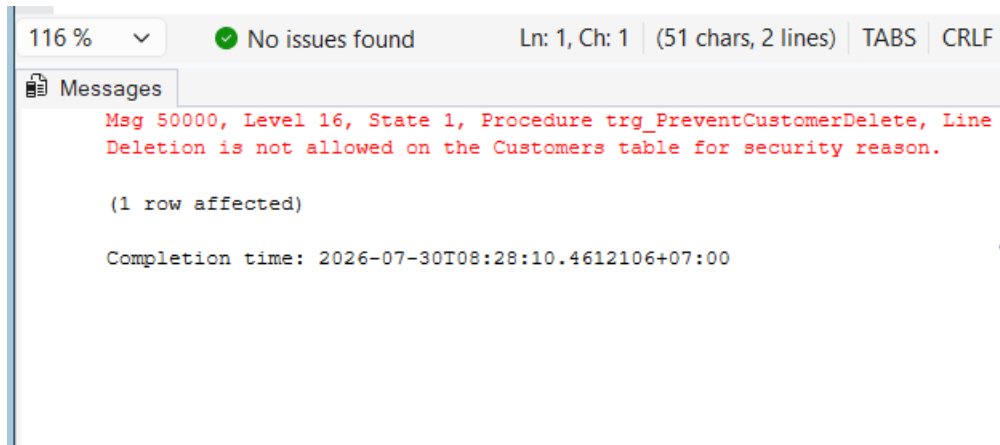
```
CREATE TRIGGER trg_PreventCustomerDelete
ON dbo.Customers
INSTEAD OF DELETE
AS
BEGIN
-- បញ្ជូនសារប្រាប់ថាការលុបត្រូវបានហាមឃាត់
    RAISERROR('Deletion is not allowed on the Customers table for security
reason.', 16, 1);
END;
GO
```

- សារបញ្ជាក់ពី Server:



- ការធ្វើតេស្តសាកល្បង (Testing)

```
DELETE FROM dbo.Customers WHERE CustomerID = 1;
GO
```



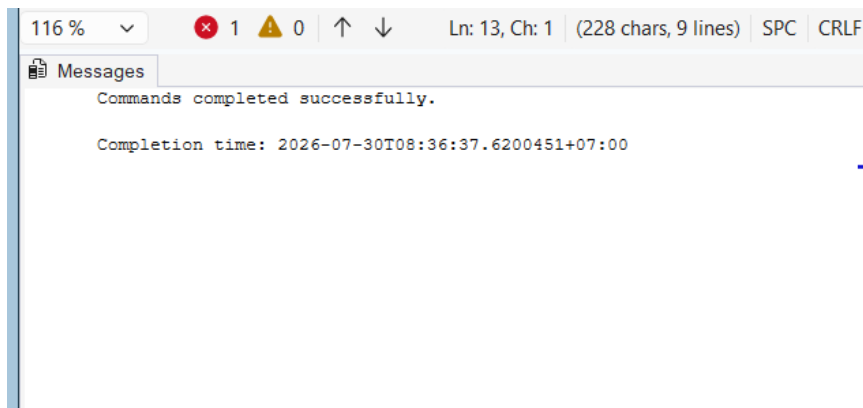
✓ **លទ្ធផល៖** Server នឹងមិនអនុញ្ញាតឱ្យលុបឡើយ ហើយនឹងបង្ហាញសារកំហុស (Error Message) ពណ៌ក្រហមនៅក្នុងផ្ទាំង Messages ថា Deletion is not allowed on the Customers table for security reasons. ដែលបង្ហាញពីអត្ថប្រយោជន៍របស់ Trigger ក្នុងការការពារទិន្នន័យ។

ឧទាហរណ៍ទី២ បង្កើត Trigger កត់ត្រា ឬធ្វើសារដំណឹងពេលមានការបញ្ចូលផលិតផលថ្មី (trg_NotifyNewProduct)

បង្កើត Trigger ប្រភេទ AFTER INSERT ដើម្បីឱ្យរាល់ពេលដែលមានការបញ្ចូលផលិតផលថ្មី (INSERT) ចូលទៅក្នុងតារាង Products Server នឹងស្វ័យប្រវត្តិបង្ហាញសារដំណឹងបញ្ជាក់ពីសកម្មភាពនោះ។

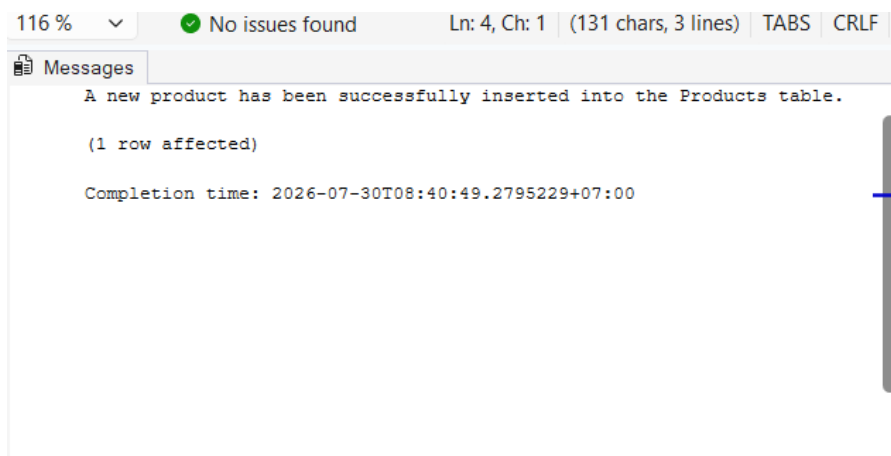
```
CREATE TRIGGER trg_NotifyNewProduct
ON dbo.Products
AFTER INSERT
AS
BEGIN
    -- បង្ហាញសារបញ្ជាក់ថាមានផលិតផលថ្មីត្រូវបានបន្ថែម
    PRINT 'A new product has been successfully inserted into the Products table.';
END;
GO
```

- សារបញ្ជាក់ពី Server:



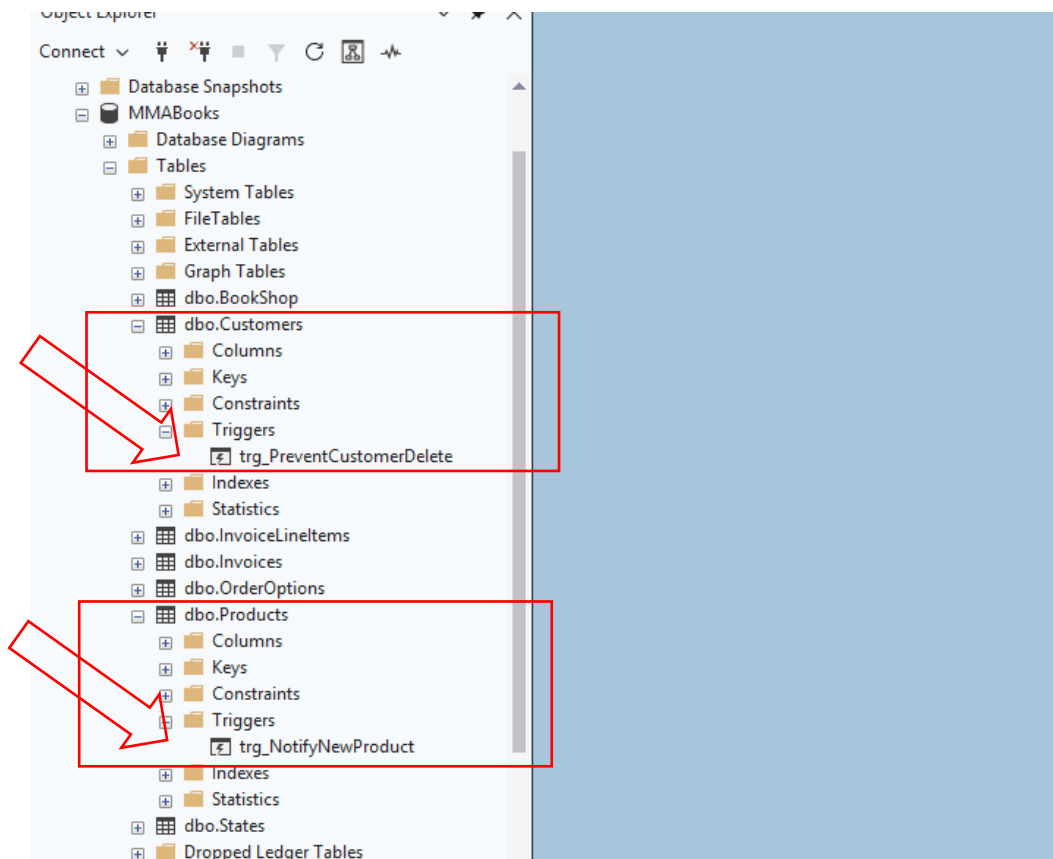
- ការធ្វើតេស្តសាកល្បង (Testing)

```
INSERT INTO dbo.Products(ProductCode,Description,UnitPrice,OnHandQuantity)
VALUES('TEST', 'Database Management Book',45.00,10);
Go
```



- ✓ **លទ្ធផល៖** ពេលចុច **Execute** ក្រៅពីការកត់ត្រាទិន្នន័យចូលតារាងពិតប្រាកដ (1 row affected) Server នឹងបង្ហាញសារបន្ថែមនៅក្នុងផ្ទាំង Messages ថា A new product has been successfully inserted into the Products table. យ៉ាងស្វ័យប្រវត្តិ។

❖ លទ្ធផលនៅលើ Server



✓ នេះគឺជាការបញ្ជាក់ចុងក្រោយថា ការបង្កើត និងរក្សាទុក Trigger ទាំងពីរនៅលើ Server គឺទទួលបានជោគជ័យ១០០ភាគរយ

IV. Reflection

✚ ចំណុចខ្លាំង (Strengths)

- **ការរចនាចរនាសម្ព័ន្ធទិន្នន័យមានលក្ខណៈស្តង់ដារ៖** ការអនុវត្តការងារតាម Report ទី១ ជួយឱ្យក្រុមការងារចេះវិភាគ និងកសាងម៉ូដែលទិន្នន័យ (ERD) យ៉ាងមានរបៀបរៀបរយ កាត់បន្ថយបញ្ហាទិន្នន័យស្ងួត (Data Redundancy) និងរក្សាបាននូវសុចរិតភាពទិន្នន័យតាមរយៈការកំណត់ Constraints យ៉ាងតឹងរឹង។
- **ការបង្កើនប្រសិទ្ធភាព និងសុវត្ថិភាពប្រព័ន្ធកម្រិតខ្ពស់៖** តាមរយៈ Report ទី២ ក្រុមការងារបានចេះប្រើប្រាស់មុខងារសំខាន់ៗដូចជា View ដើម្បីលាក់ភាពស្មុគស្មាញនៃ Query និងការពារទិន្នន័យរើសប ព្រមទាំងការប្រើប្រាស់ Stored Procedures ដើម្បីកាត់បន្ថយចរាចរណ៍ទិន្នន័យរវាង Server និង Application Layers។
- **កិច្ចសហការ និងការអនុវត្តការងារជាក្រុម៖** ការចែករំលែកគ្នាទៅវិញទៅមកក្នុងការរៀបចំប្រព័ន្ធ ធ្វើឱ្យសមាជិកម្នាក់ៗទទួលបានបទពិសោធន៍ជាក់ស្តែងទាំងផ្នែក Database Design និង Database Optimization។

✚ ចំណុចខ្សោយ និងបញ្ហាប្រឈម (Weaknesses & Challenges)

- **កង្វះបទពិសោធន៍លើការដោះស្រាយកំហុសស្មុគស្មាញ (Complex Troubleshooting)៖** ក្នុងដំណាក់កាលដំបូងនៃការសរសេរកូដ SQL កម្រិតខ្ពស់ ឬការកំណត់រចនាសម្ព័ន្ធ View និង Stored Procedures ក្រុមការងារជួបប្រទះនូវការដោះស្រាយ Error ដែលទាមទារពេលវេលាស្វែងយល់បន្ថែម។
- **ដែនកំណត់ផ្នែកហេដ្ឋារចនាសម្ព័ន្ធ៖** ការអនុវត្តប្រព័ន្ធក្នុងបរិយាកាសជាក់ស្តែងនៅតែងាយរងគ្រោះដោយសារបញ្ហាបណ្តាញ (Network Dependency) និងលក្ខណៈខ្សោយនៃម៉ាស៊ីន (Single Point of Failure) ដែលអាចប៉ះពាល់ដល់ដំណើរការទាំងមូលប្រសិនបើគ្មានផែនការបម្រុងទុកឱ្យបានល្អ។

V. សេចក្តីសន្និដ្ឋាន

សរុបជាមួយមក ការប្រើប្រាស់ View, Stored Procedures and Triggers នេះ បានបង្ហាញឱ្យឃើញយ៉ាងច្បាស់អំពីសារៈសំខាន់ និងការអនុវត្តជាក់ស្តែងនៃការប្រើប្រាស់វិធានការកម្រិតខ្ពស់នៅលើប្រព័ន្ធគ្រប់គ្រងមូលដ្ឋានទិន្នន័យ (DBMS) ។

តាមរយៈការអនុវត្តផ្ទាល់លើ SQL Server ក្រុមការងារបានយល់ដឹង និងទទួលបានបទពិសោធន៍សំខាន់ៗដូចខាងក្រោម៖

- **ការប្រើប្រាស់ View:** ជួយសម្រួលដល់ការទាញយកទិន្នន័យស្មុគស្មាញពីតារាងច្រើន (JOIN) ឱ្យមកជាតារាងនិម្មិតតែមួយ ព្រមទាំងជួយបង្កើនកម្រិតសុវត្ថិភាពដោយការលាក់ទិន្នន័យរសើប និងកំណត់សិទ្ធិអ្នកប្រើប្រាស់បានយ៉ាងល្អិតល្អន់។
- **ការប្រើប្រាស់ Stored Procedures:** បានជួយកាត់បន្ថយចរាចរណ៍ទិន្នន័យរវាង Server និង Application layers បង្កើនល្បឿនដំណើរការតាមរយៈការ Compile ទុកជាមុន ព្រមទាំងងាយស្រួលក្នុងការគ្រប់គ្រងកូដ និងធានាភាពត្រឹមត្រូវនៃប្រតិបត្តិការទិន្នន័យ។
- **ការប្រើប្រាស់ Triggers:** បានផ្តល់នូវស្វ័យប្រវត្តិកម្មកម្រិតខ្ពស់ក្នុងការគ្រប់គ្រង និងការពារសុចរិតភាពទិន្នន័យ (Data Integrity) តាមរយៈការទប់ស្កាត់សកម្មភាពដែលមិនមានការអនុញ្ញាត (ឧទាហរណ៍ ការលុបទិន្នន័យដោយអចេតនា) និងការកត់ត្រាប្រវត្តិព្រឹត្តិការណ៍នានាក្នុងប្រព័ន្ធ។

VI. Future Work (ទិសដៅការងារនាពេលអនាគត)

បន្ទាប់ពីបានសិក្សា និងអនុវត្តរបាយការណ៍ទាំង ៣ ខ្ញុំមានបំណងបន្តអភិវឌ្ឍខ្លួនក្នុងវិស័យ Database និងចង់អភិវឌ្ឍខ្លួនទៅជាអ្នកអភិវឌ្ឍន៍មូលដ្ឋានទិន្នន័យ (Database Developer) នៅពេលអនាគត។ ដើម្បីអាចធ្វើការងារនេះបានល្អ ខ្ញុំត្រូវបន្តសិក្សា និងពង្រឹងចំណេះដឹងអំពី SQL, Database Design, Database Security និង Database Optimization ព្រមទាំងការរចនា និងគ្រប់គ្រង Database ឱ្យមានប្រសិទ្ធភាព។

លើសពីនេះ ខ្ញុំត្រូវសិក្សាបន្ថែមអំពីការភ្ជាប់ Database ជាមួយ Web Application និងការប្រើប្រាស់ Programming Language ដើម្បីអាចអភិវឌ្ឍប្រព័ន្ធដែលមាន Database បាន។ ខ្ញុំក៏នឹងបន្តអនុវត្តជាមួយ

Project ជាកំស្តែង ដើម្បីបង្កើនបទពិសោធន៍ និងអាចយកចំណេះដឹងដែលបានរៀនទៅប្រើប្រាស់ក្នុងការងារពិត
ប្រាកដនាពេលអនាគត។